



Laboratorio No.2 Semana #13 – API Secrets y autenticación

Nombre: Diego Licardié **Carné:** 1251825

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

Parte 1 – Probar autenticación con Postman

Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

1. Crear la solicitud de inicio de sesión

- Abra Postman y cree una nueva colección llamada: MIA – Semana 13.
- Cree una solicitud llamada: Login.
- Seleccione el método POST.
- Utilice el endpoint: `https://dummyjson.com/auth/login`
- En Body seleccione raw y JSON.
Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.
`{ "username": "emilys", "password": "emilyspass" }`
- Ejecute la solicitud y localice en la respuesta el campo `accessToken`.

2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: `https://dummyjson.com/user/me`
- En Authorization seleccione Bearer Token.
- Copie el `accessToken` obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.

3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.
- Registre el código HTTP y explique la diferencia.

4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del `accessToken` y su envío mediante el encabezado Authorization: Bearer.

Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada: **AZURE_STORAGE_CONNECTION_STRING**
La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
`AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING`
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- No publique el archivo .env en GitHub**, ya que contiene la credencial real.

8. Crear el Secret en Github

- Ingresa al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: `AZURE_STORAGE_CONNECTION_STRING`
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.

The screenshot displays the Postman application interface. At the top, the workspace is named "Diego LC's Workspace". The left sidebar shows a collection named "url_mia" with a request named "Login". The main panel shows a POST request to "https://dummyjson.com/auth/login". The request body is a JSON object with "username": "emilys" and "password": "emilyspass". The response is a 200 OK status with a response body containing an access token, refresh token, and user information.



2.

The screenshot shows a REST client interface with the following details:

- URL:** `https://dummyjson.com/users`
- Method:** GET
- Status:** 200 OK
- Body:** A JSON array of user objects. The first object is:

```
{  "id": 1,  "firstName": "John",  "lastName": "Doe",  "username": "johnd",  "age": 34,  "gender": "male",  "email": "john.doe@dummyjson.com",  "phone": "+1 848-432-3456",  "usernames": "john",  "password": "john123",  "latitude": "39.8333",  "longitude": "http://dummyjson.com/long/lat/lat",  "height": 185.24,  "weight": 84.24,  "eyeColor": "brown",  "hair": {    "color": "brown",    "type": "curly"  },  "ip": "192.168.1.1",  "website": "http://dummy.com"}
```

3.

The screenshot shows a REST client interface with the following details:

- URL:** `https://dummyjson.com/users`
- Method:** GET
- Status:** 401 Unauthorized
- Body:** An empty JSON object:

```
{}
```



4. Función del bearer token

es una credencial temporal que se obtiene al iniciar sesión y se envía en el header Authorization: Bearer <token> en cada solicitud. Así el servidor identifica y autoriza al usuario sin que tenga que mandar usuario y contraseña cada vez.

5 y 6

```
using System;
using System.IO;
using System.Threading.Tasks;
using Azure.Storage.Blobs;
using Azure.Storage.Blobs.Models;

...

namespace MIA_AzureBlob
{
    ...

    class Program
    {
        // Nombre del contenedor en minúsculas
        no commands
        private const string containerName = "mia-archivos";

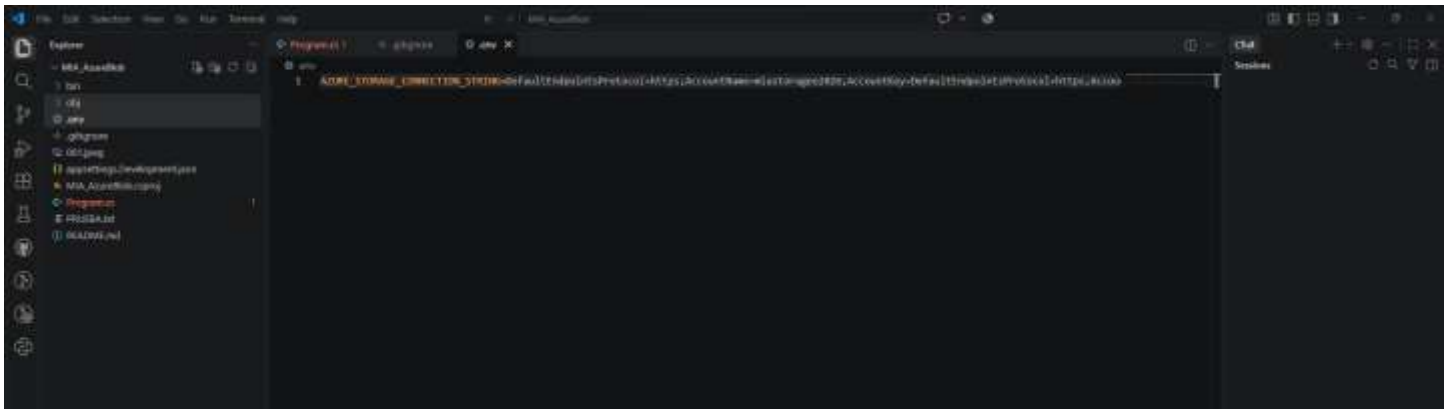
        no commands
        static async Task Main(string[] args)
        {
            // Carga las variables que estan en el archivo .env
            DotNetEnv.Env.Load();

            // La Connection String se lee desde una variable de entorno, no esta en el codigo
            string? connectionString = Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");

            if (string.IsNullOrEmpty(connectionString))
            {

```

7.





8.

The screenshot shows the GitHub Actions 'Secrets and variables' page for the repository 'Diego35647890'. The left sidebar contains a navigation menu with options like 'General', 'Actions', 'Environments', 'Repository secrets', and 'Security and quality'. The main content area is titled 'Actions secrets and variables' and includes a description of secrets and variables. Below this, there are sections for 'Environment secrets' and 'Repository secrets'. The 'Repository secrets' section shows a secret named 'L7' with the value 'ADRI_XORRAY_COMMITTON_STREAM'. A green button labeled 'New repository secret' is located at the top right of the 'Repository secrets' section.